Лабораторна робота №1

Тема: Розробка мережевого клієнт-серверного додатку з використанням сокетного інтерфейсу (Socket API) для протоколів TCP та UDP

Мета роботи:

1. Освоїти базові принципи мережевого програмування та роботу з сокетами Берклі (BSD Sockets API).
2. Навчитися реалізовувати мережеву комунікацію за допомогою протоколів транспортного рівня **TCP** (з блокуючими/неблокуючими сокетами) та **UDP**.
3. Реалізувати власний прикладний протокол обміну даними між клієнтом та сервером.
4. Дослідити поведінку сокетів при мультипотоковій обробці запитів та втраті пакетів.

Обладнання та ПЗ:

- Середовище розробки (IDE) на вибір (VS Code, PyCharm, CLion, Visual Studio, IntelliJ IDEA тощо).
- Мова програмування на вибір (Python, C/C++, C#, Java, Go, JS).
- Консольні утиліти для тестування (netcat/nc, telnet, curl).
- Мережевий аналізатор **Wireshark** (для перевірки трафіку).

1. ТЕОРЕТИЧНІ ВІДОМОСТІ (КОРОТКО)

Сокет (Socket) — це абстракція операційної системи, що представляє кінцеву точку комунікації в мережі. Сокет визначається 4-кортежем: (Source IP, Source Port, Destination IP, Destination Port).

- **TCP Sockets (SOCK_STREAM):** Забезпечують орієнтований на з'єднання потік байтів із гарантією доставки та збереженням порядку.
  - *Основні системні виклики сервера:* socket() -> bind() -> listen() -> accept() -> recv() / send() -> close().
  - *Основні системні виклики клієнта:* socket() -> connect() -> send() / recv() -> close().
- **UDP Sockets (SOCK_DGRAM):** Забезпечують передачу дискретних датаграм без встановлення з'єднання та гарантій доставки.
  - *Основні системні виклики сервера/клієнта:* socket() -> bind() (для сервера) ->

recvfrom() / sendto()-> close().

2. ЗАВДАННЯ ДО ВИКОНАННЯ

Робота складається з двох основних частин (Базова та Розширена) та Варіанта завдання відповідно до порядкового номера у списку навчальної групи.

Частина 1. Базова реалізація (TCP Echo/Command Server)

1. **Реалізуйте TCP-сервер:**
   - Сервер повинен слухати визначений порт (наприклад, 8080 або 9000).
   - Сервер має підтримувати **паралельну обробку кількох клієнтів** (за допомогою багатопотоковості multithreading, мультипроцесорності або асинхронного I/O asyncio/epoll/select).
   - При підключенні нового клієнта сервер логує в консоль: IP-адресу клієнта, ефемерний порт та час підключення.
2. **Реалізуйте TCP-клієнт:**
   - Клієнт підключається до сервера за IP-адресою та портом.
   - Зчитує команди/повідомлення з консолі користувача та надсилає їх на сервер.
   - Отримує відповідь від сервера та виводить її у консоль.
   - Підтримує команду EXIT для коректного розриву з'єднання (close()).

Частина 2. Завдання за варіантами (Прикладний логічний рівень)

Реалізуйте обробку відповідної бізнес-логіки на боці сервера за варіантом:

№	Назва системи / сервісу	Функціонал TCP-сервера (Запити/Відповіді)	Функціонал UDP-модуля
1	**Математичний нод**	Приймає вираз (наприклад, `15 + 4 * 2`), повертає обчислений результат.	**UDP Healthcheck: за запитом** `PING` **повертає** `PONG` **та Uptime сервера.**

2	**Мережевий словник**	Запит `TRANSLATE EN-UA word` $\rightarrow$ повертає переклад або помилку `404 NOT_FOUND`.	**UDP Stat: повертає кількість оброблених слів.**
3	**Конвертер валют**	Запит `CONVERT 100 USD EUR` -> повертає підраховану суму за курсом.	**UDP Rate Broadcaster: раз на 5 сек розсилає поточний курс усім слухачам.**
4	**In-Memory Key-Value БД**	Підтримує `SET key val`, `GET key`, `DELETE key` -> повертає статус або значення.	**UDP Metric: повертає кількість активних ключів у пам'яті.**
5	**Реєстр студентів**	Запит `ADD_STUDENT PIB Group`, `GET_GROUP Group` ->повертає список студентів.	**UDP Broadcast Alert: сповіщення про нові події/заняття.**
6	**Генератор паролів та хешів**	Запит `GEN_PASS 16` або `HASH_SHA256 text` -> повертає згенерований результат.	**UDP Ping/Pong з вимірюванням затримки (RTT) на боці клієнта.**
7	**Сервер часу та часових поясів**	Запит `GET_TIME Europe/Kyiv` -> повертає відформатований поточний час.	**UDP Time Sync: відправка точного UNIX timestamp для**

			синхронізації.
8	**Мережевий Note-Pad (Нотатки)**	Запит `ADD_NOTE Title Text`, `READ_NOTE Title` -> збереження та зчитування нотаток.	**UDP Storage Capacity: повертає обсяг використаної пам'яті у байтах.**
9	**Сервіс перевірки доступності IP**	Запит `CHECK_HOST 8.8.8.8` -> перевірка порту та повернення статусу `UP/DOWN`.	**UDP Fast Ping: експрес-перевірка наявності зв'язку.**
10	**Сервер валідації JSON / XML**	Приймає текстову структуру даних -> повертає `VALID` або синтаксичну помилку.	**UDP Error Counter: кількість виявлених помилок у документах.**
11	**Система черги квитків (Booking)**	Запит `BOOK_SEAT SeatNo`, `GET_FREE` -> бронювання та повернення вільних місць.	**UDP Seat Broadcaster: періодичне розсилання кількості вільних місць.**
12	**Аналізатор тексту**	Приймає текст -> повертає кількість слів, символів та найчастіше вживані слова.	**UDP Length Checker: швидка відповідь з довжиною прийнятого рядка.**

13	**Калькулятор систем числення**	Запит `CONVERT_BASE 255 10 16` -> переведення чисел між системами (2, 8, 10, 16).	**UDP System Status: інформація про мову розробки та ОС сервера.**
14	**Мережевий Todo-List**	Запит `TASK_ADD Text`, `TASK_DONE Id`, `TASK_LIST` -> керування списком завдань.	**UDP Unfinished Tasks: повернути кількість незавершених тасок.**
15	**Сервіс скорочення посилань**	Запит `SHORTEN http://...` $\rightarrow$ повертає корткий ID. `RESOLVE ID` -> повертає URL.	**UDP Redirect Counter: загальна кількість переходів за посиланнями.**
16	**Сервер Weather Mock**	Запит `GET_WEATHER City` -> повертає сформовану температуру, вологість та вітер.	**UDP Emergency Alert: розсилання штормових попереджень.**
17	**Конвертер одиниць вимірювання**	Запит `CONVERT 100 KM MILES` або `CONVERT 36.6 C F` -> повертає результат.	**UDP Supported Units: повертає список усіх підтримуваних одиниць.**
18	**Система обліку**	Запит `INVENTORY_ADD Item Qty`, `INVENTORY_GET Item` -> робота із	**UDP Low Stock Alert: розсилання сповіщень про**

	складу	залишками товарів.	**закінчення товару.**
19	**Генератор тестових даних (Faker)**	Запит `GEN_USER`, `GEN_CARD`, `GEN_EMAIL` -> повертає згенеровані фейкові дані.	**UDP Usage Counter: повертає кількість згенерованих об'єктів.**
20	**Мережева чат-кімната (Broadcaster)**	Запит `MSG User Text` -> пересилає повідомлення усім активним TCP-сесіям.	**UDP Online Count: швидкий запит кількості активних користувачів.**
21	**DNS Mock Сервер**	Запит `RESOLVE_DOMAIN example.com` -> повертає прив'язану IP-адресу.	**UDP Quick Lookup: аналог DNS-запиту за 1 датаграму.**
22	**Сервер оцінювання паролів**	Приймає рядок пароля -> повертає бал міцності (0–100%) та поради щодо покращення.	**UDP Leak Checker: перевірка, чи є пароль у списку найпоширеніших.**
23	**Система логування подій (Log Hub)**	Запит `LOG_WRITE Level Text` -> запис логу. `LOG_READ Level` $\rightarrow$ вигрузка.	**UDP Fast Log: асинхронний запис логів через UDP без чекання відповіді.**

24	**Сервер розрахунку кредитів**	Запит `CALC_LOAN Amount Months Rate` -> розрахунок щомісячного платежу.	**UDP Max Loan Limit: повертає поточну максимальну суму кредитування.**
25	**Система розумного будинку (IoT Hub)**	Запит `SET_SENSOR SensorID Value`, `GET_SENSOR SensorID` -> керування датчиками.	**UDP Telemetry Ingestion: швидкий прийом телеметрії від IoT-пристроїв.**

Частина 3. Реалізація UDP-модуля та дослідження втрат

1. Додайте до вашого сервера можливість прийому даних по **UDP** відповідно до вашого варіанта.
2. Створіть окремий **UDP-клієнт**, який надсилає 100 датаграм поспіль із порядковими номерами: Packet #1, Packet #2 ... Packet #100.
3. Реалізуйте штучну затримку або моделювання втрати пакетів на боці сервера (наприклад, відкидання кожного 10-го пакета за допомогою random).
4. На боці клієнта реалізуйте **таймаут очікування відповіді (Socket Timeout)**: якщо відповідь не прийшла протягом 1 секунди, клієнт фіксує втрату пакета і рухається далі.
5. Підрахуйте підсумковий відсоток втрачених UDP-пакетів (Packet Loss Ratio).

3. ПОКРОКОВИЙ ПЛАН ВИКОНАННЯ РОБОТИ

1. **Крок 1:** Створіть проект та опублікуйте початковий код у Git-репозиторій.
2. **Крок 2:** Напишіть базовий блокуючий TCP-сервер та перевірте його роботу за допомогою консольної утиліти netcat (або nc):
   Bash
   nc 127.0.0.1 8080
3. **Крок 3:** Додайте багатопотоковість (threading/pthreads/Task), щоб сервер міг одночасно приймати з'єднання від 3–4 різних терміналів.

4. **Крок 4:** Напишіть свій TCP-клієнт та імплементуйте бізнес-логіку за варіантом.
5. **Крок 5:** Реалізуйте UDP-сервер/клієнт та виміряйте рівень втрат пакетів при встановленні таймауту.
6. **Крок 6:** Запустіть **Wireshark**, виконайте декілька запитів і зробіть скріншоти 3-Way Handshake для TCP та окремої датаграми UDP.

4. КОНТРОЛЬНІ ЗАПИТАННЯ

1. Чим системний виклик accept() відрізняється від listen() на TCP-сервері? Що саме повертає accept()?
2. Чому для UDP-сервера не потрібно виконувати виклики listen() та accept()?
3. Що таке **Socket Exhaustion** (вичерпання сокетів) і як воно пов'язане зі станом TIME_WAIT?
4. Що станеться з TCP-клієнтом, якщо сервер несподівано зупинить процес (наприклад, за допомогою kill -9) під час активного з'єднання? Який прапор надішле ядро ОС?
5. Як реалізувати неблокуючий сокет (Non-blocking Socket) або таймаут для виклику recv()?
6. У чому полягає різниця між додаванням заголовочних структур у потік байтів TCP та обробкою меж повідомлень (Framing problem)?

5. ВИМОГИ ДО ОФОРМЛЕННЯ ЗВІТУ

Звіт оформлюється у форматі PDF і повинен містити:

1. **Тему та мету роботи**.
2. **Короткий опис розробленого прикладного протоколу** (формат запитів та відповідей між клієнтом і сервером).
3. **Вихідний код** програми (з коментарями до ключових ділянок сокетного API).
4. **Скріншоти роботи:**
   - Одночасна робота декількох клієнтів із сервером.
   - Виконання логіки за варіантом.
   - Результати тестування UDP-клієнта з демонстрацією таймаутів та втрати пакетів.
   - Дамп даного трафіку з **Wireshark** (з фіксацією прапорів TCP/UDP).
5. **Висновки** за результатами розробки.